Group 3

AI Generated Arduino Code Review

This review will be about the simplified procedures during the generation of our AI **Arduino** codes. The codes we planned to make was a code designed to read the time input for by our photoelectric gate, using a few base rules, pointed out by our prompt that has been shown below.

```
Make me an arduino code that uses pin 2 & 3 to receive input from two
photoelectric shutter. By comparing how much time a mass require to fall through
those two detectors, while remembering to give me the output in millisecond.
Pin 2 will be the input of sensor 1 and pin 3 will be sensor 2. Give me the
output as:

 1. Time for mass to be detected by sensor 1 - time when detector 1 lost the
input (passed through the sensor)

2. The same as No 1, but with sensor two instead.

3. The total time needed by the mass to pass through two sensors ( first time
a dectector pass the second sensor-the first sensor. We will be using this code
to record how long a mass will fall to find it's velocity.

Fix any of the logic that is already presented if they're any issue.
```

This generates a complicated way to do the algorism. The AI generated a code that is full of the *volatile* type variables and mainly uses the "*for*" and *"if" operators*. They've also dedicated a function for *boolean* operation which is used for sensor input detection. One for sensor 1, one for sensor 2. Both of which contains the algorithm for many types of scenarios, which honestly too "complete" for a simple algorithm. The main function is again, equipped with extra variables which is really unnecessary. This at this point just slows down the execution speed. The console's output is also really strange, and we found out the *SerialBegin()*'s value is generated differently to our standards. Here's the full code.

```
const uint8_t SENSOR1_PIN = 2; // INT0 on Uno
const uint8_t SENSOR2_PIN = 3; // INT1 on Uno

volatile unsigned long s1_start_us = 0;
volatile unsigned long s1_end_us   = 0;
volatile bool s1_blocked = false;
volatile bool s1_seen = false;

volatile unsigned long s2_start_us = 0;
volatile unsigned long s2_end_us   = 0;
volatile bool s2_blocked = false;
volatile bool s2_seen = false;

volatile bool s1_leading_captured = false;
volatile bool s2_leading_captured = false;
```

Group 3

```
void IRAM_ATTR isr_sensor1() {
  // read stable state

  bool state = digitalRead(SENSOR1_PIN); // assume sensor is HIGH when idle, LOW when blocked
  unsigned long t = micros();
  if (!state && !s1_blocked) {
    // leading edge (became blocked)
    s1_blocked = true;
    s1_start_us = t;
    s1_leading_captured = true;
  } else if (state && s1_blocked) {
    // trailing edge (unblocked)
    s1_blocked = false;
    s1_end_us = t;
    s1_seen = true;
  }
}

void IRAM_ATTR isr_sensor2() {
  bool state = digitalRead(SENSOR2_PIN);
  unsigned long t = micros();
  if (!state && !s2_blocked) {
    s2_blocked = true;
    s2_start_us = t;
    s2_leading_captured = true;
  } else if (state && s2_blocked) {
    s2_blocked = false;
    s2_end_us = t;
    s2_seen = true;
  }
}

void setup() {
  Serial.begin(115200);
  delay(100);
  Serial.println(F("Starting sensors on pins 2 (S1) and 3 (S2).")); 
  pinMode(SENSOR1_PIN, INPUT_PULLUP); // assumes sensor pulls LOW when blocked
  pinMode(SENSOR2_PIN, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(SENSOR1_PIN), isr_sensor1, CHANGE);
  attachInterrupt(digitalPinToInterrupt(SENSOR2_PIN), isr_sensor2, CHANGE);

  Serial.println(F("Waiting for events...")); 
}

void loop() {
  // When both events happen we print results, then reset flags for next measurement.
  if (s1_seen || s2_seen || (s1_leading_captured && s2_leading_captured)) {
    // copy volatile values into locals atomically
    noInterrupts();
    unsigned long s1s = s1_start_us, s1e = s1_end_us;
    unsigned long s2s = s2_start_us, s2e = s2_end_us;
    bool seen1 = s1_seen, seen2 = s2_seen;
    bool s1lead = s1_leading_captured, s2lead = s2_leading_captured;
    // clear for next
    s1_seen = false; s2_seen = false;
    s1_leading_captured = false; s2_leading_captured = false;
    interrupts();

    if (seen1 && s1e > s1s) {
      float dur1_ms = (s1e - s1s) / 1000.0f;
      Serial.print(F("Sensor1 blocked duration (ms): "));
      Serial.println(dur1_ms, 4);
    }
```

```
    if (seen2 && s2e > s2s) {
      float dur2_ms = (s2e - s2s) / 1000.0f;
      Serial.print(F("Sensor2 blocked duration (ms): "));
      Serial.println(dur2_ms, 4);
    }

    // Transit time: if both leading edges captured, compute s2_start - s1_start (expected positive)
    if (s1lead && s2lead && s2s >= s1s) {
      float transit_ms = (s2s - s1s) / 1000.0f;
      Serial.print(F("Transit time S1->S2 (ms): "));
      Serial.println(transit_ms, 6);
    } else if (s1lead && s2lead) {
      // possible if object passed S2 before S1 (unlikely for falling mass but handle)
      float transit_ms = (s1s - s2s) / 1000.0f;
      Serial.print(F("Transit time (S2->S1) (ms) [unexpected order]: "));
      Serial.println(transit_ms, 6);
    }

    Serial.println(F("--- ready for next object ---"));
  }
}
```

It's indeed a long code, designed to be prepared for **everything**. But the output is strange, and by strange we mean:

1. Repeating output of sensor 1 and 2 which results in a confusing data range.
2. The time between is in microseconds.
3. Strange time intervals as a output.
4. Garbled texts. (*which we already debugged by changing it to 9600 baud*)

The output from sensor 1 and 2 keeps **spamming** the control bar, flooding it with unconsistent readings, which in turns caused the interval between the two sensors to be extremely skewed. To fix it, we told it to make a limiter for the outputs, which in turn did decreased the output amount. But still, a very crowded control bar and the interval problem has yet to be fixed. This is how we phrased it:

```
Add a stop switch. Stop the sensors to detect, so they stop outputing inputs.
```

This simple prompt changed the code drasticly, but still the issue still persists. The AI just adds a few new extra function for this, namely: *void attachSensorInterrupts(),*and *void detachSensorInterruptsAndClear()*.

```
void attachSensorInterrupts() {
  noInterrupts();
  attachInterrupt(digitalPinToInterrupt(SENSOR1_PIN),
sensor1ISR, CHANGE);
  attachInterrupt(digitalPinToInterrupt(SENSOR2_PIN),
sensor2ISR, CHANGE);
  // clear any stale data
s1_start_us = 0; s1_end_us = 0; s1_blocked = false;
s1_done = false;
s2_start_us = 0; s2_end_us = 0; s2_blocked = false;
s2_done = false;
sensorsEnabled = true;
interrupts();
}
```

```
void detachSensorInterruptsAndClear() {
  noInterrupts();
detachInterrupt(digitalPinToInterrupt(SENSOR1_PIN));
detachInterrupt(digitalPinToInterrupt(SENSOR2_PIN));

s1_start_us = 0; s1_end_us = 0; s1_blocked = false;
s1_done = false;
s2_start_us = 0; s2_end_us = 0; s2_blocked = false;
s2_done = false;
  sensorsEnabled = false;
  interrupts();
```

We don't really felt the effect these functions gives, so we told the AI to really try to fix the problem, simplify it further, and to get rid of all those extra function and variables. At this point, the script is around 260++ words and unreliable. This is what we put a the prompt:

> Please really simplify it. No need for those extra scenario. use digitalRead instead. Throw all those function and variables. SIMPLIFY IT.

This did wonders! Not only this massive reduced the output number, but also really simplified it. Less codes, less chances to break! It overalls feels stabler, and it almost looks identically as the ones given by the assistant teachers.

```
const int S1 = 2;
const int S2 = 3;
boolean s1_blocked = false;
boolean s2_blocked = false;
unsigned long s1_start = 0;
unsigned long s1_end   = 0;
unsigned long s2_start = 0;
unsigned long s2_end   = 0;

void setup() {
  Serial.begin(9600);
  pinMode(S1, INPUT_PULLUP);
  pinMode(S2, INPUT_PULLUP);
  Serial.println("Simplified detection with booleans (ms). S1=2, S2=3");
}

void loop() {
  // --- Sensor 1 ---
  if (!s1_blocked && digitalRead(S1) == LOW) {
    s1_blocked = true;
    s1_start = millis();   // object enters S1
  }

  if (s1_blocked && digitalRead(S1) == HIGH) {
    s1_end = millis();     // object leaves S1
  }

  // --- Sensor 2 ---
  if (!s2_blocked && digitalRead(S2) == LOW) {
    s2_blocked = true;
    s2_start = millis();   // object enters S2
  }

  if (s2_blocked && digitalRead(S2) == HIGH) {
    s2_end = millis();     // object leaves S2
  }

  // --- When both sensors are finished ---
  if (s1_end > s1_start && s2_end > s2_start) {
    unsigned long s1_dur = s1_end - s1_start;
    unsigned long s2_dur = s2_end - s2_start;
    long transit = (long)s2_start - (long)s1_start;

    Serial.print("1) S1 blocked duration: ");
```

```
Serial.print(s1_dur);
  Serial.println(" ms");

  Serial.print("2) S2 blocked duration: ");
  Serial.print(s2_dur);


  Serial.print("3) Transit S1->S2: ");
  Serial.print(transit);
  Serial.println(" ms");

  Serial.println("--- ready for next object ---");

  // Reset for next measurement
  s1_blocked = false;
  s2_blocked = false;
  s1_start = s1_end = s2_start = s2_end = 0;

  delay(200); // small pause
 }
}
```

And yet, the data we get is quite unique. They're really far from the theoretical supposed values, but really stable. Stable meaning, the outputs are roughly the same each time we time it. It cannot compare to those machines that are designed to do this kinds of things, far outranks them. But with time and effort, it can be designed to compete with those instruments.

In conclusion, AI indeed can make an Arduino code, but with the right prompt and knowledge, it can do great. But still, we prefer to think AI as a tool to help us build the codes. Human Ingeniuty still plays a huge role in make outstanding codes and tools. AI is just a programs with hundreds of thousands conversation etched into their systems, every thing they output are based on already existing conversation which don't really count as creative. We are responsible for that part. Creativity. Humans must learn to coexist with AI, not depend on them. With all that said, we'll like to say our thanks for reading.

Well Regards,
Group 3